

In []:

```
__nbid__ = '0053'  
__author__ = 'Eric Armengaud, Benjamin Weaver <benjamin.weaver@noirlab.edu>  
__version__ = '20260505' # yyyymmdd  
__datasets__ = ['boss_dr17', 'desi_dr1', 'sdss_dr17']  
__keywords__ = ['sparcl', 'spectroscopy', 'sdss spectra', 'desi spectra', '']
```

Obtain and plot spectra data using SPARCL, prospect, and specutils

Credit: [Eric Armengaud](#), Saclay - CEA, is the primary author of prospect. See also the [prospect contributors](#). [Benjamin Weaver](#) and [Alice Jacques](#) contributed to the development of this notebook.

Table of contents

- [Summary & goals](#)
- [Disclaimer & attribution](#)
- [Imports & setup](#)
- Find, retrieve, and plot spectra from:
 - [DESI](#)
 - [SDSS](#)
 - [\(e\)BOSS](#)

Summary & goals

[prospect](#) is an interactive spectrum visualization tool created by the [DESI collaboration](#) to view and characterize DESI spectra. It has also been adapted to display data from other projects, specifically the [Sloan Digital Sky Survey \(SDSS\)](#).

Project: Obtain DESI DR1, SDSS DR17, and (e)BOSS DR17 spectra using the [NOIRLab SPARCL spectrum service](#), convert data to [specutils objects](#) as needed, and use [prospect](#) to display the data.

Takeaway: At no point in this notebook will any data files be opened, except for an "Aside" section.

Takeaway: prospect allows multiple, independent spectra visualizations to coexist within

This notebook is an adaptation of the [Prospect + specutils + SPARCL](#) notebook. The figure below shows a portion of the prospect display with an example DESI spectrum: the observed spectrum is plotted in red (smoothed by a small Gaussian kernel with $\sigma = 1$), the best-fit pipeline model in black and the uncertainty in green (stored as the inverse variance `ivar`). DESI and SDSS spectra further include a `mask` array indicating if some pixels are problematic (in which case `mask!=0`) and SDSS spectra include a `sky` spectrum, which is not shown nor used in this notebook. The prospect tool also includes a color image at the location of the fiber and the interactive tool allows the user to click on the thumbnail image to open the [Legacy Surveys Viewer](#) at that sky location in a separate browser tab.



Note that using the Astro Data Lab constitutes your agreement with our minimal [Disclaimers](#).

If you use **Astro Data Lab** in your published research, please include the text in your paper's Acknowledgments section:

This research uses services or data provided by the Astro Data Lab, which is part of the Community Science and Data Center (CSDC) Program of NSF NOIRLab. NOIRLab is operated by the Association of Universities for Research in Astronomy (AURA), Inc. under a cooperative agreement with the U.S. National Science Foundation.

If you use **SPARCL jointly with the Astro Data Lab platform** (via JupyterLab, command-line, or web interface) in your published research, please include this text below in your paper's Acknowledgments section:

This research uses services or data provided by the SPectra Analysis and Retrievable Catalog Lab (SPARCL) and the Astro Data Lab, which are both part of the Community Science and Data Center (CSDC) Program of NSF NOIRLab. NOIRLab is operated by the Association of Universities for Research in Astronomy (AURA), Inc. under a cooperative agreement with the U.S. National Science Foundation.

In either case **please cite the following papers**:

- Data Lab concept paper: Fitzpatrick et al., "The NOAO Data Laboratory: a conceptual overview", SPIE, 9149, 2014, <https://doi.org/10.1117/12.2057445>
- Astro Data Lab overview: Nikutta et al., "Data Lab - A Community Science Platform", Astronomy and Computing, 33, 2020, <https://doi.org/10.1016/j.ascom.2020.100411>

If you are referring to the Data Lab JupyterLab / Jupyter Notebooks, cite:

- Juneau et al., "Jupyter-Enabled Astrophysical Analysis Using Data-Proximate Computing Platforms", CiSE, 23, 15, 2021, <https://doi.org/10.1109/MCSE.2021.3057097>

If publishing in a AAS journal, also add the keyword: `\facility{Astro Data Lab}`

And if you are using SPARCL, please also add `\software{SPARCL}` and cite:

- Juneau et al., "SPARCL: SPectra Analysis and Retrievable Catalog Lab", Conference Proceedings for ADASS XXXIII, 2024 <https://doi.org/10.48550/arXiv.2401.05576>

The NOIRLab Library maintains [lists of proper acknowledgments](#) to use when publishing papers using the Lab's facilities, data, or services.

For this notebook specifically, please acknowledge:

- prospect: [Eric Armengaud](#), Saclay - CEA, the primary author of prospect. See also the [prospect contributors](#).

Imports & setup

Note that we only lightly using the DESI software stack (DESI 25.3), although some

imports will be embedded within prospect .

```
In [ ]: import os
import sys
sys.path.insert(1, os.path.join(os.environ['HOME'], 'Documents', 'Code', 'g
from numba import NumbaDeprecationWarning
import warnings
warnings.filterwarnings("ignore", category=NumbaDeprecationWarning, message
import numpy as np
import matplotlib.pyplot as plt
import astropy.units as u
from astropy.nddata import InverseVariance
from astropy.io import fits
from specutils import __version__ as specutils_version, Spectrum1D
from bokeh import __version__ as bokeh_version
from prospect import __version__ as prospect_version
from prospect.viewer import plotspectra
from prospect.specutils import Spectra
from desispec import __version__ as desispec_version
from desispec.io import findfile
from redrock import __version__ as redrock_version
from redrock.templates import get_template_dir, load_templates
from sparcl import __version__ as sparcl_version
from sparcl.client import SparclClient
from dl import __version__ as dl_version, queryClient as qc, authClient as
print(f"astro-datalab=={dl_version}")
print(f"specutils=={specutils_version}")
print(f"bokeh=={bokeh_version}")
print(f"prospect=={prospect_version}")
print(f"desispec=={desispec_version}")
print(f"redrock=={redrock_version}")
print(f"sparcl=={sparcl_version}")
```

Authentication

Much of the functionality of Data Lab can be accessed without explicitly logging in (the service then uses an anonymous login). But some capacities, for instance saving the results of your queries to your virtual storage space, require a login (*i.e.* you will need a registered user account).

If you need to log in to Data Lab, un-comment the cell below and execute it:

```
In [ ]: # from getpass import getpass
# token = ac.login(input("Enter user name: (+ENTER) "), getpass("Enter pass
ac.whoAmI())
```

Start SPARCL Client

```
In [ ]: client = SparclClient()  
client
```

Set up Data Lab database interface

All data should be visible in the 'default' profile

```
In [ ]: qc.get_profile()
```

DESI DR1

Find DESI spectra

SPARCL provides access to DESI spectra that have been coadded by HEALPixel. This corresponds to entries in the `desi_dr1.zpix` table at Astro Data Lab (see column information [here](#)).

```
In [ ]: q = """  
SELECT z.targetid, z.chi2, z.z, z.zerr, z.zwarn, z.spectype, z.subtype,  
       z.coadd_numexp, z.coadd_exptime, z.healpix, z.deltachi2  
FROM desi_dr1.zpix AS z  
WHERE z.zcat_primary  
      AND z.survey = 'main'  
      AND z.program = 'dark'  
      AND z.spectype = 'GALAXY'  
      AND z.z BETWEEN 0.5 AND 0.9  
      AND z.zwarn = 0  
ORDER BY z.targetid  
LIMIT 50  
"""  
desi_ids = qc.query(sql=q, fmt='table')  
desi_ids
```

QA: Do we really have 50 unique spectra?

`assert` guarantees that the expression being evaluated is `True`, and is used in many standard tests in Python.

```
In [ ]:
```

```
assert (np.unique(desi_ids['targetid']) == desi_ids['targetid']).all()
```

Retrieve DESI spectra

With the set of `targetid` obtained above, we can directly retrieve DESI spectra.

```
In [ ]: include = client.get_all_fields(dataset_list=['DESI-DR1'])
desi_spectra = client.retrieve_by_specid(specid_list=desi_ids['targetid'].value,
                                         include=include,
                                         dataset_list=['DESI-DR1'])

desi_spectra.info
```

QA: Did we really find all of the expected spectra?

Side effect: this also sorts the returned records to match the order of the inputs.

```
In [ ]: desi_spectra = desi_spectra.reorder(desi_ids['targetid'].value.tolist())
assert (np.array([r.targetid for r in desi_spectra.records]) == desi_ids['targetid'].value).all()
assert all([r.survey == 'main' for r in desi_spectra.records])
assert all([r.program == 'dark' for r in desi_spectra.records])
```

QA: Do all spectra records have the same wavelength solution?

```
In [ ]: assert all([(r.wavelength == desi_spectra.records[0].wavelength).all() for r in desi_spectra.records])
```

QA: Are any of the spectra fully masked?

```
In [ ]: desi_fully_masked = np.zeros((len(des_i_spectra.records), ), dtype=bool)

for k in range(len(des_i_spectra.records)):
    assert np.isfinite(des_i_spectra.records[k].flux).all()
    assert np.isfinite(des_i_spectra.records[k].ivar).all()
    if (des_i_spectra.records[k].mask > 0).all():
        print(f"WARNING: Spectrum record {k:d} is fully masked!")
        desi_fully_masked[k] = True
```

View the contents of the first retrieved record

```
In [ ]: desi_spectra.records[0]
```

Organize metadata

Prospect needs several inputs:

1. An object containing spectra. In this case we'll use `prospect.specutils.Spectra`, which inherits from `SpectrumList`, and is really just a `Spectrum1D` object underneath.
 - The object contains the usual flux, wavelength, uncertainty.
 - In addition a "fibermap" table is needed. This should be an Astropy `Table` with the expected columns.
2. A redshift catalog. This should be an Astropy `Table` with the expected columns.
3. A model spectrum. The model is actually provided by SPARCL, but we need to input it separately.

Spectrum object

First we assemble the components of the spectrum object. Most of the work is done simply by converting the retrieved results to a `specutils` object.

```
In [ ]: desi_specutils = desi_spectra.to_specutils()
```

We need a "fibermap" table. We'll start with photometric quantities.

```
In [ ]: columns = ('targetid', 'ra', 'dec', 'ref_epoch', 'pmra', 'pmdec', 'ebv',
                  'flux_g', 'flux_r', 'flux_z', 'flux_w1', 'flux_w2')
q = """
SELECT {0}
FROM desi_dr1.photometry
WHERE targetid IN ({1})
ORDER BY targetid
""".format(',', '.join(columns), ', '.join([str(t) for t in desi_ids['targetid']]))

fibermap = qc.query(sql=q, fmt='table')
```

Prospect expects upper-case columns, so we convert that here.

```
In [ ]: for col in fibermap.colnames:
        if col == 'ra' or col == 'dec':
            fibermap.rename_column(col, 'TARGET_' + col.upper())
        else:
            fibermap.rename_column(col, col.upper())
```

QA: Did we find photometry for every `targetid` ?

```
In [ ]: assert (fibermap['TARGETID'] == desi_ids['targetid']).all()
```

Next we add targeting bitmasks, which encode information on how objects were targeted and are defined in the [DESI Data Model documentation](#).

```
In [ ]: columns = ('targetid', 'desi_target', 'bgs_target', 'mws_target', 'scnd_target')
q = """
SELECT DISTINCT {0}
FROM desi_dr1.target
WHERE targetid IN ({1})
ORDER BY targetid
""".format(', '.join(columns), ', '.join([str(t) for t in fibermap['TARGETID']]))

targeting = qc.query(sql=q, fmt='table')
```

Convert column names to upper-case.

```
In [ ]: for col in targeting.colnames:
        targeting.rename_column(col, col.upper())
```

QA: Did we find targeting for every targetid ?

```
In [ ]: assert (targeting['TARGETID'] == fibermap['TARGETID']).all()
```

QA: Does the order of fibermap match the SPARCL records?

```
In [ ]: assert (np.array([r.targetid for r in desi_spectra.records]) == fibermap['TARGETID']).all()
```

Add columns into fibermap .

```
In [ ]: for col in ('DESI_TARGET', 'BGS_TARGET', 'MWS_TARGET', 'SCND_TARGET'):
        fibermap.add_column(targeting[col])
        fibermap
```

Finally assemble the object. Although we already created a `specutils.Spectrum` object above, Prospect expects a slightly different version of a `specutils` object.

```
In [ ]: desi_prospect = Spectra(bands=['coadd'],
                               wave={'coadd': desi_specutils.spectral_axis.value},
                               flux={'coadd': desi_specutils.flux[~desi_fully_mask]})
```

```
ivar={'coadd': desi_specutils.uncertainty[~desi_fullmask],
mask={'coadd': desi_specutils.mask[~desi_fully_masked],
fibermap=fibermap[~desi_fully_masked],
meta={'coadd': desi_specutils.meta}}
```

Redshift catalog

We can re-use the initial query above; it was deliberately constructed.

```
In [ ]: desi_zcatalog = desi_ids.copy()

# prospect expects the column names in upper case letters and the HEALPIX column
# to instead be named HPXPIXEL so we make these changes below
for col in desi_zcatalog.colnames:
    if col == 'healpix':
        desi_zcatalog.rename_column(col, 'HPXPIXEL')
    else:
        desi_zcatalog.rename_column(col, col.upper())
desi_zcatalog
```

Model spectra

Prospect expects a model in the form of a tuple containing wavelength and flux. Since SPARCL provides the model, this is easy. There are other ways to specify the model, but these require more access to the DESI software stack *and data files*.

```
In [ ]: desi_model = (desi_specutils.spectral_axis, desi_specutils.meta['model'][~d
```

Start prospect

With everything assembled, the interface to prospect is just a single call. Note that we're setting `with_vi_widgets=True` in order to fully describe the prospect display. Other visualization examples below will have `with_vi_widgets=False`.

Further details about the prospect display:

- The large window displays a spectrum with best fit model overlaid.
- Scanning the mouse over that plot shows a full-resolution zoom of that portion of the spectrum to the right, which is particularly useful for scanning narrow emission lines.
- Try the various tools at the top for panning, zooming, selecting. When the wheel zoom tool is selected (default), using the scroll wheel will zoom in and out. Point the

mouse over the x or y axis before scrolling to zoom on just that axis.

- Click on one of the legend entries to the lower right to turn that item on or off.
- The example below includes visual inspection (VI) tools, in a shaded box to the lower left. This allows individuals to rate spectra on questions such as "is the redshift accurate?", or "is the object really a galaxy as opposed to a star?". Guidelines on using the VI tools are just below the tools themselves.
- Immediately above the VI tools are arrows and a slider for displaying other spectra.
- To the right of the VI tools are metadata about the spectrum, interspersed with tools for smoothing the spectrum, manually changing the redshift of the best-fit model, and toggling the display of emission and absorption line markers.

```
In [ ]: plotspectra(desiprospect,
                    zcatalog=desi_zcatalog[~desi_fully_masked],
                    redrock_cat=None,
                    notebook=True,
                    with_thumb_tab=False,
                    with_vi_widgets=True,
                    with_coaddcam=False,
                    mask_type='DESI_TARGET',
                    model_from_zcat=False,
                    model=desi_model)
```

Aside: Model spectra in DESI DR1

The models displayed along with the spectra in Prospect are assembled from a set of templates. [Redrock](#) performs a best-fit and the resulting model is a linear combination of some of the templates, depending on the final type of the object (galaxy, QSO, etc.). In some cases it can be useful to reconstruct the models using software from redrock. In this section we'll take a look at how to perform this reconstruction.

Some of the code below is adapted from an [in-depth tutorial](#) on redrock.

Load redrock templates

This cell simply identifies the location of the template files.

```
In [ ]: specprod = 'iron'
         template_dir = get_template_dir()
         templates = load_templates(f'{template_dir}/templates-{specprod}.txt', asdi
```

Find a set of QSOs

There are two different sets of QSO templates, L0Z and HIZ, corresponding to different subtypes for QSO spectypes. This query will return a set of spectra with both L0Z and HIZ subtypes. In addition, the coefficients of the template model are retrieved, e.g. `z.coeff_0`.

```
In [ ]: q = """
SELECT z.targetid, z.chi2, z.z, z.zerr, z.zwarn, z.spectype, z.subtype,
       z.coadd_numexp, z.coadd_exptime, z.healpix, z.deltachi2,
       z.coeff_0, z.coeff_1, z.coeff_2, z.coeff_3, z.coeff_4,
       z.coeff_5, z.coeff_6, z.coeff_7, z.coeff_8, z.coeff_9
FROM desi_dr1.zpix AS z
WHERE z.zcat_primary
      AND z.survey = 'main'
      AND z.program = 'dark'
      AND z.spectype = 'QSO'
      AND z.healpix = 26067
      AND z.z BETWEEN 1.0 AND 4.0
      AND z.zwarn = 0
ORDER BY z.targetid
LIMIT 50
"""
desi_qso_ids = qc.query(sql=q, fmt='table')
desi_qso_ids
```

Retrieve the spectra and models from SPARCL

```
In [ ]: include = client.get_all_fields(dataset_list=['DESI-DR1'])
desi_qso_spectra = client.retrieve_by_specid(specid_list=desi_qso_ids['targetid'],
                                             include=include,
                                             dataset_list=['DESI-DR1'])
desi_qso_spectra.info
```

Verify that we retrieved all the expected spectra, and sort the results to match the order of the inputs.

```
In [ ]: desi_qso_spectra = desi_qso_spectra.reorder(desi_qso_ids['targetid'].value)
assert (np.array([r.targetid for r in desi_qso_spectra.records if r.survey
```

```
In [ ]: desi_qso_specutils = desi_qso_spectra.to_specutils()
```

Combine coefficients and templates to get the redrock models

```
In [ ]: assert (desi_qso_ids['spectype'] == 'QSO').all()
```

```

assert ((desi_qso_ids['subtype'] == 'HIZ') | (desi_qso_ids['subtype'] == 'L
qso_loz = templates[('QSO', 'LOZ')]
qso_hiz = templates[('QSO', 'HIZ')]
tmpl_model_flux = np.zeros((len(desi_qso_ids), len(desi_qso_specutils.spect
for i, row in enumerate(desi_qso_ids):
    row_coeff = np.array([row[f'coeff_{j:d}'] for j in range(10)])
    if row['subtype'] == 'LOZ':
        tmpl_model_flux[i, :] = qso_loz.eval(row_coeff, desi_qso_specutils.
    else:
        tmpl_model_flux[i, :] = qso_hiz.eval(row_coeff, desi_qso_specutils.

```

Comparison plot

```

In [ ]: fig = plt.figure(figsize=(16, 9), dpi=100)
ax = fig.add_subplot(111)
spectrum_index = 24 # Step through spectra here.
pl0 = ax.plot(desi_qso_specutils.spectral_axis, desi_qso_specutils.flux[spe
pl1 = ax.plot(desi_qso_specutils.spectral_axis, desi_qso_specutils.meta['mo
pl2 = ax.plot(desi_qso_specutils.spectral_axis, tmpl_model_flux[spectrum_in
txt = ax.set_xlabel('Wavelength [Å]')
txt = ax.set_ylabel(r'Flux [ $10^{-17}$  \mathrm{erg} \, \, \mathrm{cm}^{-2} \, \mat
txt = ax.set_title(f"TARGETID={desi_qso_ids[spectrum_index]['targetid']:d};
l = ax.legend()

```

SDSS

Find SDSS spectra

```

In [ ]: q = """
SELECT z.specobjid, z.bestobjid, z.z, z.zerr, z.zwarning, z.class, z.subcla
        z.rchi2diff, z.primtarget, z.sectarget
FROM sdss_dr17.specobj AS z
WHERE z.bestobjid > 0
      AND z.run2d = '26'
      AND z.plate = 2955
      AND z.mjd = 54562
      AND z.class = 'GALAXY'
      AND z.zwarning = 0
ORDER BY z.specobjid
LIMIT 50
"""

sdss_ids = qc.query(sql=q, fmt='table')
sdss_ids

```

QA: Do we really have 50 unique spectra?

```
In [ ]: assert (np.unique(sdss_ids['specobjid']) == sdss_ids['specobjid']).all()
```

Retrieve SDSS spectra

With the set of `specobjid` obtained above, we can directly retrieve SDSS spectra.

```
In [ ]: include = client.get_all_fields(dataset_list=['SDSS-DR17'])
sdss_spectra = client.retrieve_by_specid(specid_list=sdss_ids['specobjid'],
                                         include=include,
                                         dataset_list=['SDSS-DR17'])

sdss_spectra.info
```

QA: Did we really find all of the expected spectra? Note that we have to assume `specid == specobjid`.

Side effect: this also sorts the returned records to match the order of the inputs.

```
In [ ]: sdss_spectra = sdss_spectra.reorder(sdss_ids['specobjid'].value.tolist())
assert (np.array([r.specid for r in sdss_spectra.records]) == sdss_ids['specobjid']).all()
assert all([r.plate == 2955 for r in sdss_spectra.records])
assert all([r.mjd == 54562 for r in sdss_spectra.records])
```

QA: Do all spectra records have the same wavelength solution?

```
In [ ]: assert all([(r.wavelength == sdss_spectra.records[0].wavelength).all() for r in sdss_spectra.records])
```

QA: Are any of the spectra fully masked?

```
In [ ]: sdss_fully_masked = np.zeros((len(sdss_spectra.records), ), dtype=bool)

for k in range(len(sdss_spectra.records)):
    assert np.isfinite(sdss_spectra.records[k].flux).all()
    assert np.isfinite(sdss_spectra.records[k].ivar).all()
    if (sdss_spectra.records[k].mask > 0).all():
        print(f"WARNING: Spectrum record {k:d} is fully masked!")
        sdss_fully_masked[k] = True
```

Note that one of the spectra has a non-zero mask for every pixel. This will cause problems with some versions of Prospect, so for the demonstration below, we will not attempt to plot that spectrum. Future versions of Prospect will handle this more gracefully.

View the contents of the first retrieved record

```
In [ ]: sdss_spectra.records[0]
```

Organize metadata

Prospect needs several inputs:

1. An object containing spectra. In this case we'll use a [Spectrum](#) object.
 - The object contains the usual flux, wavelength, uncertainty.
 - In addition a "plugmap" table is needed. This should be an Astropy Table with the expected columns.
2. A redshift catalog. This should be an Astropy Table with the expected columns.
3. A model spectrum. The model is actually provided by SPARCL, but we need to input it separately.

Spectrum object

First we assemble the components of the spectrum object. Most of the work is done simply by converting the retrieved results to a `specutils` object.

```
In [ ]: sdss_specutils = sdss_spectra.to_specutils()
```

We need a "plugmap" table. We'll start with photometric quantities.

```
In [ ]: columns = ('z.specobjid', 'p.objid', 'p.ra', 'p.dec', 'p.u', 'p.g', 'p.r',
q = ""
SELECT {0}
FROM sdss_dr17.specobj AS z
JOIN sdss_dr17.photoplate AS p
ON z.bestobjid = p.objid
WHERE p.objid IN ({1})
ORDER BY z.specobjid
"".format(', '.join(columns), ', '.join([str(t) for t in sdss_ids['bestobj

plugmap = qc.query(sql=q, fmt='table')
```

```
In [ ]: for col in plugmap.colnames:
    plugmap.rename_column(col, col.upper())
mag = np.zeros((len(plugmap), 5), dtype=plugmap['G'].value.dtype)
for k, band in enumerate('UGRIZ'):
```

```
mag[:, k] = plugmap[band].value
plugmap.add_column(mag, name='MAG')
plugmap.add_column(sdss_ids['primgtarget'], name='PRIMTARGET')
plugmap.add_column(sdss_ids['sectarget'], name='SECTARGET')
sdss_specutils.meta['plugmap'] = plugmap[~sdss_fully_masked]
```

QA: Did we find photometry for every bestobjid ?

```
In [ ]: assert (plugmap['OBJID'] == sdss_ids['bestobjid']).all()
```

Finally assemble the object.

```
In [ ]: sdss_prospect = Spectrum1D(flux=sdss_specutils.flux[~sdss_fully_masked, :],
                                   spectral_axis=sdss_specutils.spectral_axis,
                                   uncertainty=InverseVariance(sdss_specutils.uncer
mask=sdss_specutils.mask[~sdss_fully_masked, :])
                                   meta=sdss_specutils.meta)
```

Redshift catalog

We can re-use the initial query above; it was deliberately constructed.

```
In [ ]: sdss_zcatalog = sdss_ids.copy()
for col in sdss_zcatalog.colnames:
    if col == 'zerr':
        sdss_zcatalog.rename_column(col, 'Z_ERR')
    else:
        sdss_zcatalog.rename_column(col, col.upper())
sdss_zcatalog
```

Model spectra

Prospect expects a model in the form of a tuple containing wavelength and flux. Since SPARCL provides the model, this is easy. There are other ways to specify the model, but these require more access to the SDSS *files*.

```
In [ ]: sdss_model = (sdss_specutils.spectral_axis, sdss_specutils.meta['model'][~s
```

Start prospect

With everything assembled, the interface to prospect is just a single call.

```
In [ ]: plotspectra(sdss_prospect,
                    zcatalog=sdss_zcatalog[~sdss_fully_masked],
                    redrock_cat=None,
                    notebook=True,
                    with_thumb_tab=False,
                    with_vi_widgets=False,
                    with_coaddcam=False,
                    mask_type='PRIMTARGET',
                    model_from_zcat=False,
                    model=sdss_model)
```

(e)BOSS

Find (e)BOSS spectra

```
In [ ]: q = """
SELECT z.specobjid, z.bestobjid, z.z, z.zerr, z.zwarning, z.class, z.subcla
        z.rchi2diff, z.boss_target1, z.eboss_target0, z.eboss_target1,
        z.eboss_target2
FROM sdss_dr17.specobj AS z
WHERE z.bestobjid > 0
      AND z.run2d = 'v5_13_2'
      AND z.plate = 9599
      AND z.mjd = 58131
      AND z.class = 'GALAXY'
      AND z.zwarning = 0
ORDER BY z.specobjid
LIMIT 50
      """
boss_ids = qc.query(sql=q, fmt='table')
boss_ids
```

QA: Do we really have 50 unique spectra?

```
In [ ]: assert (np.unique(boss_ids['specobjid']) == boss_ids['specobjid']).all()
```

Retrieve (e)BOSS spectra

With the set of `specobjid` obtained above, we can directly retrieve (e)BOSS spectra.

```
In [ ]: include = client.get_all_fields(dataset_list=['BOSS-DR17'])
boss_spectra = client.retrieve_by_specid(specid_list=boss_ids['specobjid'],
                                         include=include,
```

```
dataset_list=['BOSS-DR17'])

boss_spectra.info
```

QA: Did we really find all of the expected spectra?

Side effect: this also sorts the returned records to match the order of the inputs.

```
In [ ]: boss_spectra = boss_spectra.reorder(boss_ids['specobjid'].value.tolist())
assert (np.array([r.specid for r in boss_spectra.records]) == boss_ids['spe
assert all([r.plate == 9599 for r in boss_spectra.records])
assert all([r.mjd == 58131 for r in boss_spectra.records])
```

QA: Do all spectra records have the same wavelength solution?

```
In [ ]: assert all([(r.wavelength == boss_spectra.records[0].wavelength).all() for
```

QA: Are any of the spectra fully masked?

```
In [ ]: boss_fully_masked = np.zeros((len(boss_spectra.records), ), dtype=bool)

for k in range(len(boss_spectra.records)):
    assert np.isfinite(boss_spectra.records[k].flux).all()
    assert np.isfinite(boss_spectra.records[k].ivar).all()
    if (boss_spectra.records[k].mask > 0).all():
        print(f"WARNING: Spectrum record {k:d} is fully masked!")
        boss_fully_masked[k] = True
```

View the contents of the first retrieved record

```
In [ ]: boss_spectra.records[0]
```

Organize metadata

Prospect needs several inputs:

1. An object containing spectra. In this case we'll use a [Spectrum1D](#) object.
 - The object contains the usual flux, wavelength, uncertainty.
 - In addition a "plugmap" table is needed. This should be an [Astropy Table](#) with the expected columns.
2. A redshift catalog. This should be an [Astropy Table](#) with the expected columns.
3. A model spectrum. The model is actually provided by SPARCL, but we need to input

it separately.

Spectrum object

First we assemble the components of the spectrum object. Most of the work is done simply by converting the retrieved results to a `specutils` object.

```
In [ ]: boss_specutils = boss_spectra.to_specutils()
```

We need a "plugmap" table. We'll start with photometric quantities.

```
In [ ]: columns = ('z.specobjid', 'p.objid', 'p.ra', 'p.dec', 'p.u', 'p.g', 'p.r',
q = ""
SELECT {0}
FROM sdss_dr17.specobj AS z
JOIN sdss_dr17.photoplate AS p
ON z.bestobjid = p.objid
WHERE p.objid IN ({1})
ORDER BY z.specobjid
"".format(', '.join(columns), ', '.join([str(t) for t in boss_ids['bestobjid']]))

plugmap = qc.query(sql=q, fmt='table')
```

```
In [ ]: for col in plugmap.colnames:
    plugmap.rename_column(col, col.upper())
mag = np.zeros((len(plugmap), 5), dtype=plugmap['G'].value.dtype)
for k, band in enumerate('UGRIZ'):
    mag[:, k] = plugmap[band].value
plugmap.add_column(mag, name='MAG')
plugmap.add_column(boss_ids['boss_target1'], name='BOSS_TARGET1')
plugmap.add_column(boss_ids['eboss_target0'], name='EBOSS_TARGET0')
plugmap.add_column(boss_ids['eboss_target1'], name='EBOSS_TARGET1')
plugmap.add_column(boss_ids['eboss_target2'], name='EBOSS_TARGET2')
boss_specutils.meta['plugmap'] = plugmap[~boss_fully_masked]
```

QA: Did we find photometry for every `bestobjid` ?

```
In [ ]: assert (plugmap['OBJID'] == boss_ids['bestobjid']).all()
```

Finally assemble the object.

```
In [ ]: boss_prospect = Spectrum1D(flux=boss_specutils.flux[~boss_fully_masked, :],
    spectral_axis=boss_specutils.spectral_axis,
    uncertainty=boss_specutils.uncertainty[~boss_fully_masked])
```

```
mask=boss_specutils.mask[~boss_fully_masked, :]  
meta=boss_specutils.meta)
```

Redshift catalog

We can re-use the initial query above; it was deliberately constructed.

```
In [ ]: boss_zcatalog = boss_ids.copy()  
for col in boss_zcatalog.colnames:  
    if col == 'zerr':  
        boss_zcatalog.rename_column(col, 'Z_ERR')  
    else:  
        boss_zcatalog.rename_column(col, col.upper())  
boss_zcatalog
```

Model spectra

Prospect expects a model in the form of a tuple containing wavelength and flux. Since SPARCL provides the model, this is easy. There are other ways to specify the model, but these require more access to the (e)BOSS files.

```
In [ ]: boss_model = (boss_specutils.spectral_axis, boss_specutils.meta['model'][~b
```

Start prospect

With everything assembled, the interface to prospect is just a single call.

```
In [ ]: plotspectra(boss_prospect,  
                    zcatalog=boss_zcatalog[~boss_fully_masked],  
                    redrock_cat=None,  
                    notebook=True,  
                    with_thumb_tab=False,  
                    with_vi_widgets=False,  
                    with_coaddcam=False,  
                    mask_type='EBOSS_TARGET1',  
                    model_from_zcat=False,  
                    model=boss_model)
```

```
In [ ]:
```